Technical Report

ABU-SSO: Unified Single Sign-On & Digital Campus Wallet

Course: COEN838 — Advanced Software Engineering
Department: Computer Engineering, Ahmadu Bello University, Zaria
Student: Idris Muhammad Abubakar
Matric No: P25EGCP9003
Date: October 2026

Repository: https://github.com/idrismakr-png/abu-sso
Live Deployment: https://abu-sso.onrender.com

Abstract

Nigerian university students routinely juggle multiple credentials across isolated systems (student portal, LMS, Wi-Fi, library, hostel), leading to password fatigue, forgotten logins, and shared accounts. This report documents the engineering of **ABU-SSO** — a unified single sign-on and digital campus wallet for Ahmadu Bello University — following the Software Engineering process taught in COEN838 (Modules I–V). The system is built with Python 3.12, FastAPI, SQLAlchemy, and JWT-based authentication with role-based access control. It was delivered in twelve iterative increments, is covered by **121 automated tests (116 fast + 5 E2E) achieving 96% line coverage**, runs a green CI pipeline on every push, has passed a Bandit SAST scan with zero findings, and is deployed live on Render (7 web services) with a Neon PostgreSQL database. The system includes six independent SSO services that trust ABU-SSO as their identity provider, demonstrating real distributed single sign-on. The report closes with a risk register, maintenance plan, and reflections.

---

1. Introduction

1.1 Problem Statement

Students at Ahmadu Bello University interact with at least six separate digital systems, each with its own credentials:

1. Central student portal (course registration)
2. Learning Management System (Moodle)
3. Campus Wi-Fi captive portal
4. Library borrowing system
5. Hostel allocation and gate access
6. Result checker

Passwords are frequently forgotten, reused across systems, or shared with peers — a security and productivity problem. Identity is duplicated across the systems, violating the "once-only" registration principle.

1.2 Goal

Design and build a **single identity provider** for ABU campus services, with a **digital wallet** for payments and **role-based administration**, exposed via a **documented REST API** so any campus service can trust one login.

1.3 Scope

In scope:

- Central authentication (matric-based login, JWT issuance).
- Role-based access control (student / staff / admin).
- **Admin-controlled account creation** (bootstrap-only first user, then admin-created accounts).
- Digital ID card with QR code.
- Campus wallet (top-up, payment, transaction history).
- Web dashboard + admin panel.
- **Six independent SSO services** — Portal, LMS, Library, Hostel, Wi-Fi, Result Checker — each deployed separately.
- Auto-generated OpenAPI documentation.
- Automated tests + continuous integration.
- Cloud deployment (Render for 7 web services + Neon PostgreSQL).

Out of scope: real payment gateways, native mobile apps, biometric authentication, and integration with the real ABU ERP.

---

2. Engineering Process

2.1 Methodology — Agile, Iterative

An **iterative and incremental Agile** process was used because the problem statement involves real users with evolving needs. Each increment produced a working, demonstrable slice of the system:

Increment	Deliverable
1	Project scaffold + Git + GitHub repo
2	FastAPI base + health endpoint
3	Modular architecture (config, routers)
4	SQLAlchemy + User model
5	bcrypt password hashing + user service
6	JWT auth endpoints
7	Digital ID card + QR
8	Campus wallet
9	Web UI (login + dashboard)
10	53 tests + GitHub Actions CI
11	RBAC + Admin panel + admin create-user + cloud deployment

Increment	Deliverable
12	Six distributed mock SSO services + 121 tests with E2E

Why not Waterfall? Wallet, admin, and distributed SSO features were added *after* the auth prototype was working — an upfront plan could not have anticipated the exact wallet schema, the admin workflow, or the number of downstream services. Iteration allowed course correction.

2.2 Requirements Engineering (Module I)

Requirements were elicited using three techniques:

- **Interviews** with 3 ABU students.
- **Ethnography** — observing students in the Faculty of Engineering computer lab.
- **Document analysis** — the ABU ICT Directorate's policy on the "once-only" principle.

19 Functional Requirements (FR-01 to FR-19) and **13 Non-Functional Requirements** (NFR-01 to NFR-13) are documented in `docs/SRS.md`. Requirements are modelled with:

- Use Case diagram (10 use cases)
- Sequence diagram (login + protected resource)
- Class diagram (User, Wallet, Transaction)

A **Requirements Traceability Matrix** (`docs/RTM.md`) links every FR to its design artefact, source file, and automated test.

2.3 Architecture & Design (Module III)

Architectural style: A **modular monolith** (the ABU-SSO identity provider) **plus six independent microservices** that trust its JWTs.

The IdP:

- Layered architecture organised around **feature routers** (`auth`, `wallet`, `id_card`, `admin`, `pages`).
- Layers: `HTTP → Router → Service → Model → Database`.

The distributed layer:

- Six independent FastAPI applications, each deployed as its own Render web service.
- Each service verifies tokens by calling the IdP's `/auth/me` endpoint — no shared session, no shared database.
- Stateless verification means any service can be scaled independently.

Inter-component communication: synchronous REST over HTTP with JWT in the `Authorization: Bearer` header.

Data model: three tables — `users`, `wallets`, `transactions`. A user owns 0 or 1 wallet; a wallet owns 0..N transactions.

Scalability: stateless JWT means the API can be horizontally scaled behind a load balancer. The database is swappable from SQLite to PostgreSQL by changing the `DATABASE_URL` environment variable — a migration we exercised in the deployment phase.

2.4 Security Engineering (Module IV)

Threat model: STRIDE was applied to each endpoint. Full analysis in `docs/SECURITY.md`.

Secure design principles applied:

Principle	Where applied
Least privilege	No user can access another user's wallet or ID card. Admin-only routes use `require_role("admin")`.
Defence in depth	bcrypt (passwords) + JWT signing (sessions) + Pydantic validation (input) + ORM parameterisation (SQLi) + RBAC (authorization) + bootstrap-only registration.
Fail securely	Invalid/expired tokens return 401; unauthorised roles return 403; insufficient funds raise `ValueError` and roll back.
Never trust input	Every request body is validated by Pydantic schemas.
Separation of secrets	`JWT_SECRET` in `.env` (gitignored); in production, Render auto-generates.
Cryptographic hygiene	bcrypt with auto-generated salt; JWT secret ≥ 32 bytes (RFC 7518).

Authentication scheme: HS256 JWT with 15-minute expiry. Tokens contain `sub` (user id), `iat`, and `exp`. Users log in with their **matric / staff number** — not email.

Authorization scheme: Role-based access control. Three roles — `student`, `staff`, `admin` — enforced by a reusable FastAPI dependency factory `require_role(*roles)`.

Account provisioning policy:

- **First user on a fresh deployment** → automatic bootstrap as admin.
- **All subsequent accounts** → created by an admin via `POST /admin/users`.
- **Public `/auth/register`** → returns **HTTP 403** once any user exists.

This models the real ABU workflow: students are registered by ICT after admission, not self-registered.

Security testing: Bandit SAST ran against `app/`. Result: **0 issues identified** across 525 lines of code.

2.5 Testing & Quality Assurance (Module II)

Testing follows a **four-layer pyramid**:

Layer	Count	Tooling	What it covers
Unit	7	pytest	`hash_password`, `verify_password`
Service	51	pytest + SQLAlchemy	`user_service`, `wallet_service`, `admin_service`
Integration	21	pytest + FastAPI TestClient	All public HTTP routes
System (E2E)	5	pytest + Playwright	Browser-driven user journeys

Layer	Count	Tooling	What it covers
Total	**84**		

Coverage: 96% of the `app/` package (473 statements, 19 missed).

Static analysis: Bandit 1.9.4 → 0 findings.

Continuous Integration: `.github/workflows/ci.yml` runs the fast suite on every push, then the E2E suite.

Full details in `docs/TEST_RESULTS.md`.

2.6 Deployment

Deployed on free-tier cloud services:

- **IdP:** Render web service (Python 3.12, Uvicorn, auto-deploy on push to `main`).
- **Six SSO services:** Render web services — `abu-sso-portal`, `abu-sso-lms`, `abu-sso-library`, `abu-sso-hostel`, `abu-sso-wifi`, `abu-sso-results`.
- **Database:** Neon (managed PostgreSQL 18, no expiry).
- **Configuration:** `render.yaml` declares all seven services; `DATABASE_URL` is set as an environment variable pointing to Neon.

Live URL: https://abu-sso.onrender.com
SSO services: https://abu-sso-portal.onrender.com, https://abu-sso-lms.onrender.com, etc.

2.7 Project Management & Maintenance (Module V)

See Sections 5, 6, and `docs/RISK_AND_MAINTENANCE.md`.

---

3. Implementation Summary

3.1 Technology Stack

Layer	Technology
Language	Python 3.12
Web framework	FastAPI 0.142
Templating	Jinja2
ORM	SQLAlchemy 2.1
Database (dev)	SQLite
Database (prod)	PostgreSQL (Neon)
Authentication	JWT (PyJWT) + bcrypt
Authorization	RBAC middleware (`require_role`)
QR codes	qrcode + Pillow

Layer	Technology
Testing	pytest + httpx + Playwright
CI	GitHub Actions
SAST	Bandit
Hosting	Render (7 services) + Neon

3.2 Project Structure

abu-sso/ ├── app/ │ ├── main.py # App factory, router registration │ ├── config.py # Settings (pydantic-settings) │ ├── database.py # SQLAlchemy engine + session │ ├── models/ # ORM: User, Wallet, Transaction │ ├── schemas/ # Pydantic request/response schemas │ ├── routers/ # auth, wallet, id_card, admin, health, pages │ ├── services/ # Business logic │ ├── utils/ # security.py, jwt.py, rbac.py │ ├── templates/ # Jinja2 HTML (login, dashboard, admin) │ └── static/ # CSS + JS ├── mock-services/ # 6 independent SSO services │ ├── portal/ # Student Portal (port 4001) │ ├── lms/ # LMS (port 4002) │ ├── library/ # Library (port 4003) │ ├── hostel/ # Hostel (port 4004) │ ├── wifi/ # Wi-Fi (port 4005) │ └── results/ # Result Checker (port 4006) ├── tests/ # 84 pytest tests ├── docs/ # SRS, RTM, Design, Security, etc. ├── render.yaml # Blueprint for 7 Render services └── .github/workflows/ci.yml # CI pipeline

3.3 Key Code Highlights

Password hashing (`app/utils/security.py`):

```
def hash_password(plain: str) -> str:
    return bcrypt.hashpw(plain.encode(), bcrypt.gensalt()).decode()

def verify_password(plain: str, hashed: str) -> bool:
    return bcrypt.checkpw(plain.encode(), hashed.encode())

   Bootstrap-only registration (app/routers/auth.py):
    @router.post("/register", response_model=UserRead, status_code=201)
def register(payload: UserCreate, db: Session = Depends(get_db)):
    if db.query(User).count() == 0:
        return create_user(db, ..., role="admin")  # bootstrap
    if settings.allow_public_registration:
        return create_user(db, ..., role=payload.role)
    raise HTTPException(403, "Public registration is disabled.")

    Auth middleware (app/utils/jwt.py):
    def get_current_user(credentials = Depends(bearer_scheme), db =
Depends(get_db)) -> User:
    user_id = decode_token(credentials.credentials) if credentials else None
    if not user_id:
        raise HTTPException(401, "Invalid or expired token")
    user = get_user_by_id(db, user_id)
    if not user or not user.is_active:
        raise HTTPException(401, "User not found or inactive")
    return user
```

```
    RBAC dependency factory (app/utils/rbac.py):
    def require_role(*allowed_roles: str):
    def _checker(current_user: User = Depends(get_current_user)) -> User:
        if current_user.role not in allowed_roles:
            raise HTTPException(403, f"Requires role: {' or
'.join(allowed_roles)}")
        return current_user
    return _checker

    Distributed SSO verification (mock-services/portal/main.py):
    async def verify_token_with_idp(token: str) -> dict:
    async with httpx.AsyncClient() as client:
        res = await client.get(
            f"{IDP_URL}/auth/me",
            headers={"Authorization": f"Bearer {token}"},
            timeout=5.0,
        )
    if res.status_code != 200:
        raise HTTPException(401, "Invalid or expired token")
    return res.json()

    4. STRIDE Threat Model
    Threat  Applies?    Where   Mitigation
Spoofing    Yes Login endpoint  bcrypt password verification; matric lookup
Tampering   Yes Request bodies  Pydantic validation; ORM parameterisation
Repudiation Yes Wallet actions  Every transaction stored with timestamp
Information Disclosure  Yes Password / token leaks  Hashes only; HTTPS; JWT never
logged; QR contains no PII
Denial of Service   Partial Public endpoints    Rate limiting planned (residual
risk)
Elevation of Privilege  Yes Cross-user and cross-role access    get_current_user
scopes reads; require_role blocks non-admins; bootstrap-only registration prevents
privilege self-assignment

Full STRIDE analysis in docs/SECURITY.md.

5. Risk Register
#   Risk    P   I   Mitigation
R1  JWT secret leaked via Git history   M   H   .env gitignored; Render generates;
≥32-byte key; rotated after chat exposure
R2  Weak user passwords H   M   Pydantic min-length; admin-set initial passwords
R3  SQLite concurrency at scale M   M   Documented PostgreSQL migration path
R4  Time overrun vs. course workload    M   H   Agile increments
R5  Exam-day network failure    L   H   Local demo also works; screen recording
backup
R6  Dependency CVE  M   M   Monthly pip-audit; Dependabot
R7  Unauthorized data access via IDOR   L   H   All reads scoped to
get_current_user
R8  Render free-tier cold start H   L   First request takes 30–60 s; documented in
README

6. Maintenance Plan
6.1 Maintenance Categories
Category    % effort    Examples
```

```
Corrective  20% Fix bugs reported by users; patch dependency CVEs
Adaptive    20% Migrate to newer Python; add OIDC; support new browsers
Perfective  50% Audit log viewer, bulk user management, rate limiting, mobile UI
Preventive  10% Monthly pip-audit; refactor hot paths; rotate JWT secret

6.2 Versioning & Release Process
Semantic versioning: MAJOR.MINOR.PATCH.

Conventional Commits: feat:, fix:, docs:, test:, chore:, ci:.

Release checklist: all tests pass locally + CI; coverage ≥ 70%; bandit -r app/
clean; .env.example current; SRS/RTM updated.

6.3 Deprecation & Migration
If the JWT algorithm changes (HS256 → RS256 for OIDC), accept both during a
transition window, then sunset HS256.

6.4 Metrics to Monitor in Production
Metric  Target
API availability    ≥ 99%
Login latency p95   ≤ 500 ms
Wallet payment failure rate < 1%
CI pass rate    100%

7. Results
Metric  Result
Working prototype   ✅ Deployed at https://abu-sso.onrender.com
Functional Requirements implemented 19 / 19
Distributed SSO services    6 (Portal, LMS, Library, Hostel, Wi-Fi, Results)
Automated tests    121 / 121 passing (116 fast + 5 E2E)
Line coverage   96%
SAST findings   0 (Bandit)
CI status   ✅ Green
UML diagrams    3 (use case, sequence, class)
Roles enforced  3 (student / staff / admin)
Documents delivered SRS, RTM, Design, Security, User Manual, Test Results, Risk &
Maintenance, Technical Report, Presentation Guide, AI Use Declaration

8. Reflections and Lessons Learned
Iteration beats planning. Wallet, admin, and distributed features were added after
auth was working; a strict Waterfall plan would have frozen the schema too early.

Tests pay for themselves. Bugs were caught by tests during each increment that
manual testing missed.

Automate early. GitHub Actions from week 1 saved hours of manual test runs.

Documentation is code. The SRS forced clarifications that improved the actual
implementation.

Security is a process. Bandit found a short JWT secret during development; without
SAST, that would have shipped. The bootstrap-only registration pattern closed a
later-discovered self-registration vulnerability.
```

Cloud complexity is manageable. Deploying to two free-tier providers (Render + Neon) was cheaper and more durable than a bundled provider.

Distributed proof is stronger than distributed theory. Building six real mock services was worth more for the demo than any architectural diagram.

9. Future Work
OIDC / OpenID Connect with RS256 asymmetric keys.

Audit log table recording all admin actions.

Rate limiting via slowapi.

Refresh tokens for better UX.

Email verification + OTP password reset.

Real payment gateway integration.

Native mobile app reusing the same API.

Offline-first QR verification for gate access.

10. References
Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill.

FastAPI documentation: https://fastapi.tiangolo.com

RFC 7519 (JWT), RFC 7518 (JWA)

OWASP Top 10 – 2021

11. Assumptions
The following assumptions were made in interpreting the exam problem statement (Question 7 – Unified ABU-SSO), as required by exam instruction D.

11.1 Scope Assumptions
Assumption Justification
A1 The system is a prototype, not a full ABU ICT production rollout. Course timescale and access to real ABU infrastructure.
A2 Integration with the real ABU portal and Moodle is out of scope; six mock services are used instead. Real integration requires vendor cooperation and ABU ICT approval.
A3 Wallet top-ups are simulated; no real payment gateway. PCI-DSS and provider onboarding are beyond course scope.

11.2 User Assumptions
Assumption Justification
A4 Every student has a matric number; every staff member has a staff number. Issued by ABU at admission/employment.
A5 Users access a modern web browser. Standard for web applications.

A6 Administrators are trusted ABU ICT staff. Standard assumption for RBAC.
A7 Students do not self-register — accounts are created by ICT after admission. Realistic ABU workflow; enforced via bootstrap-only registration.

11.3 Technical Assumptions
Assumption Justification
A8 SQLite is sufficient for dev/demo; PostgreSQL for production. SQLite avoids setup friction; PostgreSQL matches cloud reality.
A9 JWT secret is delivered via environment variable. Standard cloud security practice.
A10 15-minute session expiry is acceptable for the demo. Refresh tokens are future work.
A11 QR codes encode only {id, matric_no, role} — no PII. Data minimisation (NDPR).
A12 Mock services verify tokens via the IdP's /auth/me, not locally with the secret. Simpler, safer, models OIDC UserInfo.
A13 Network availability exists between browser and cloud. Standard web assumption.

11.4 Business / Operational Assumptions
Assumption Justification
A14 Wallet balances represent campus currency (₦). Matches "digital campus wallet" scope.
A15 Horizontal scaling is documented but not exercised. Prototype scale.
A16 NDPR audit is not required for the prototype. No real student PII stored.

11.5 Limitations Explicitly Acknowledged
No rate limiting on /auth/login — planned via slowapi.

No refresh tokens — 15-minute re-login required.

No email verification — moot, since email isn't used for authentication.

No audit log table yet — admin actions not persisted to dedicated trail.

Mock services are demonstrations — real services would call the same JWT-protected API.

Render free-tier cold starts — first request after idle takes 30–60 s.

Registration is bootstrap-only — matches ABU reality, but means admin access is required to onboard new users.

11.6 Compliance Statement
These assumptions were reviewed against the problem statement to ensure no requirement was silently dropped. All assumptions are also reflected in docs/SRS.md §2.6 and in the risk register.

12. AI Use Declaration
In accordance with COEN838 exam instruction F ("Ensure responsible and ethical use of AI"), the full disclosure of AI assistance in this project is provided in a standalone document:
📄 docs/AI_USE_DECLARATION.md

```
Summary: Anthropic's Claude was used as a conversational coding tutor — assisting
with environment setup, explaining error messages, suggesting patterns, and
reviewing documentation structure. All engineering decisions, architectural
choices, testing strategy, deployment plan, and written narrative are the
candidate's own work. Every AI-suggested fix was verified by running the code and
inspecting the result.
```